

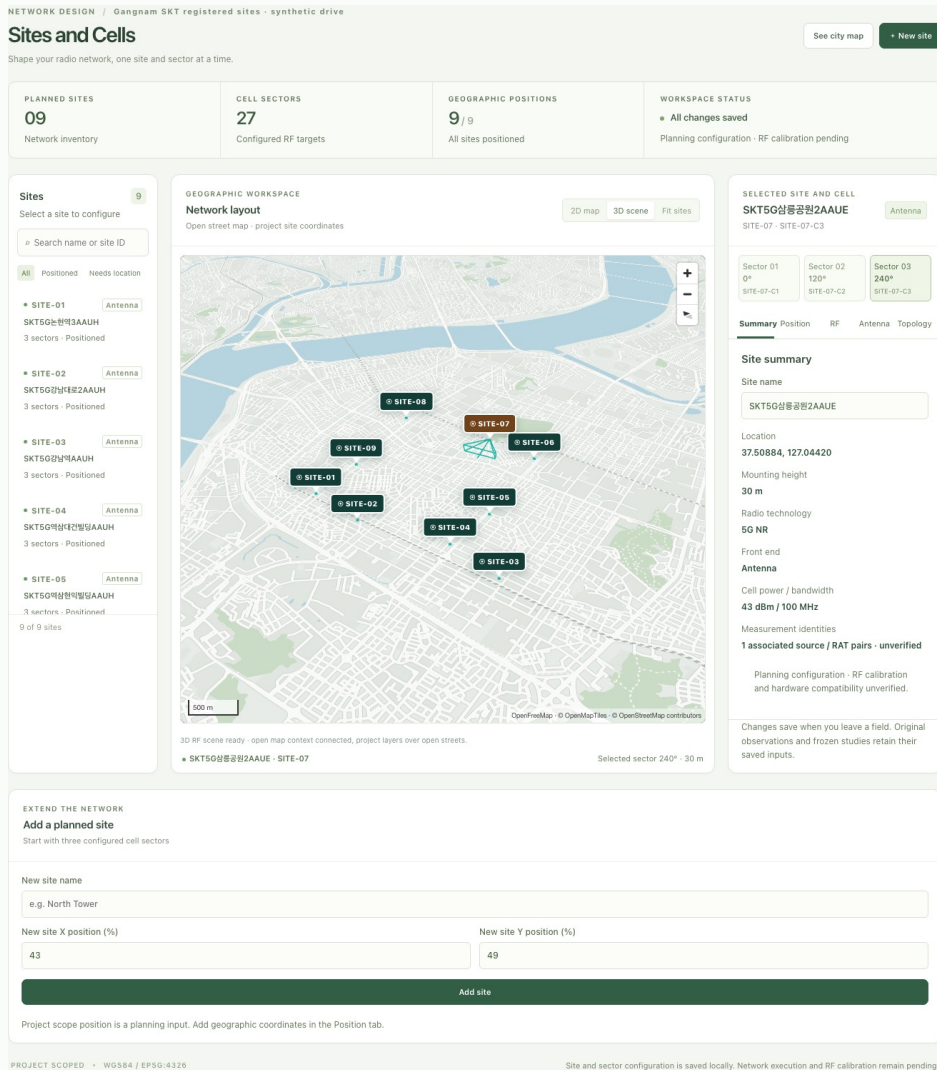
Atlas RAN Twin

Development guide

Code dependencies, Graft, SQLite and the developer workflow

Repository working-tree snapshot
2 October 2026

Project scope



Implemented locally

React workspaces, WGS84 scenes, measurement evidence, SQLite and retained studies.

Optional execution

A bounded Sionna-RT path worker runs in a separate Python environment.

Further integration

Live RAN adapters, RF calibration and GH200 resource control need additional implementation.

Engineering workflow

The navigation follows the engineering task. Projects provide the shared scope

Workspace section	Main routes	Development concern
Overview	overview	Context, readiness and source status
Data and Twin Setup	study, measurements, map	Definitions, evidence and geometry
Network Design	planner, radio, scenarios	Site/cell edits and retained alternatives
Simulation and Experiments	ray, ab, data	Captured inputs and execution boundaries
Validation and Optimization	drive	KPI availability, sample analysis and provenance
Operations and Reports	runs, artifacts, tasks, schedule	Run history, exports and planned work

Code ownership by layer

Start in the layer that owns the behavior, then trace its callers with Graft

Layer	Starting files	Responsibility
React shell	src/main.tsx src/app/ and src/legacy/ActivityPreview.tsx	Entry, routing, layout and global project context
Feature workspaces	src/features/<area>/	Rendering, local sessions and command adapters
Domain rules	model.mjs, workspaces.mjs dm.mjs, study.mjs, measurement library	Pure transformations, validation and evidence identity
Transport	src/api/ serve.py	JSON contracts, HTTP boundaries and job orchestration
Persistence and worker	storage.py, *_storage.py rt_worker.py	SQLite transactions, retained records and local solving

Runtime dependency chain

A persisted feature change crosses these interfaces in order

Owner	Depends on	Contract at the boundary
React feature	Feature command + local session	Validated user intent and project ID
AppController.dispatch	Domain helpers + workspaceSchema	Immutable workspace snapshot
workspaceApi.writeWorkspace	Artifact index + fetch + schemas	PUT { revision, workspace, artifacts }
serve.py handler	ProjectStore	HTTP 200, 400 or 409 result
storage.py	sqlite3 + evidence validators	Atomic revisioned transaction
Optional path execution	JobManager + rt_worker.py	Frozen request and retained run evidence

Pinned package dependencies

The lockfile defines the install. The launcher resolves the project-local tools

Package	Declared version	Purpose
React / React DOM	19.3.0 / 19.3.0	Rendering and subscriptions
MapLibre GL	5.23.0	Open 2D/3D geographic context
Zod	4.6.5	Runtime transport parsing
TypeScript	5.9.3	Strict frontend type checks
Vite / React plugin	8.3.1 / 6.1.1	Development proxy and production build
Vitest / jsdom	4.1.11 / 27.2.0	Component and API tests
Playwright	1.63.0	Chromium workflow tests

Development environment

Tool	Repository requirement / configuration	Observed locally
Node.js	^22.12.0 or >=24.0.0 CI uses Node 24	26.10.0
Python + SQLite	Python 3.10+ with stdlib sqlite3 CI uses Python 3.13	Python 3.14.7
npm	Use the checked-in package-lock.json	12.1.0
Graft CLI	Separate installation on PATH	0.21.1
Optional RT	uv + Python 3.13 + Sionna-RT 2.1.0 Compatible native backend required	Check separately with test:rt

First local run

Run these commands from the checkout root on macOS, Linux or Windows

1. Install the pinned dependencies

Include the optional native bindings required by the current platform.

`npm run setup`

`npm start`

2. Build and start the application

The launcher serves the React bundle and Python/SQLite API together.

`npm run help`

3. Use the server's printed URL

The normal address is `http://127.0.0.1:8765/`.
Automatic fallback can select another port.

Windows: `npm.cmd` works when PowerShell blocks `npm.ps1`.
The base app needs no additional Python packages.

Frontend development and local API

Terminal A: Python/SQLite API

`npm run api`

Terminal B: Vite frontend

`npm run dev`

Setting	Meaning
ATLAS_API_ORIGIN	Vite proxy target. Default: <code>http://127.0.0.1:8765</code>
ATLAS_DB_PATH	SQLite file location. Use a separate file for development fixtures.
ATLAS_PYTHON	Python executable path, without additional arguments
PLAYWRIGHT_PORT	Browser-test Vite port. Default: 5174

Graft orientation

AGENTS.md requires graph context before source searches or edits

```
graft map
graft ask "KPI trend sample sequence" --source
graft skeleton src/app/AppController.ts
graft callers dispatch
graft grep "dm-trend"
```

Install the team's approved Graft version separately.
Observed tool: @nanonets/graft 0.21.1.

Dependency tracing with Graft

Question	Command	Use the result
Where is the behavior?	<code>graft ask "commitDriveImport" --source</code>	Open only the relevant returned spans
What does it depend on?	<code>graft callers commitDriveImport --direction out</code>	Identify the domain and persistence path
Who will a change affect?	<code>graft callers dispatch --depth 2</code>	Review callers and their contracts
Where are all occurrences?	<code>graft grep "dm-trend"</code>	Search every indexed match
Is the graph current?	<code>graft build</code> <code>graft check</code>	Refresh and verify local context

React entry points and route wiring

File	Role when adding a workspace
src/main.tsx	Mount App under StrictMode and load shared styles
src/app/App.tsx	Choose the initial route and preview launcher behavior
src/legacy/ActivityPreview.tsx	Own the controller, select the feature route and wire actions
src/app/routeRegistry.ts	Register the stable route ID and URL aliases
src/app/workflowNavigation.ts	Place the workspace in the engineering menu
src/app/PreviewWorkspaceShell.tsx	Connect shell layout, context and navigation

Route IDs remain usable through ?workspace=drive and the registered aliases.

State ownership

Persist engineering inputs through ApplicationController. Keep view preferences in feature sessions

State	Owner	Lifetime
Projects, plans, evidence, studies	AppController + ProjectStore	SQLite workspace snapshot
Map extent, selected sample/site, layers	ProjectMapSession	Current owner/project/scope session
Drive playback and analysis selection	DriveSession	Transient view state
Artifacts and run query results	AppController query state	Refreshable project-scoped cache
Confirmed-save browser backup	AppController.backup	Migration source only when SQLite is empty

Revised save pipeline

Step	Owner	Behavior
1	Feature command	Create a project-scoped transformation
2	AppController.dispatch	Clone, validate and queue the new workspace
3	workspaceApi.writeWorkspace	Verify evidence and submit the current revision
4	ProjectStore.save_workspace	BEGIN IMMEDIATE, compare revision and write atomically
5	Controller confirmation	Advance revision and back up the confirmed save
Conflict	HTTP 409 / WorkspaceConflictError	Keep the local draft and block further queued saves

Read-only comparison and an explicit reload decision help resolve conflicting tabs.

SQLite schema

Table	Key / relationship	Stored evidence
workspace_meta	Singleton id = 1	Active project and global workspace revision
projects	id	Identity, status and project_json
activity	project_id + position	Ordered project activity
tasks	project_id + task_id	Planned task payload and schedule
artifacts	project_id + artifact_id	Generated/exportable file content
task_runs	id, project_id, retry_of	Captured request, result and lifecycle
run_events	run_id + sequence	Ordered state transitions

Studies and measurement versions remain inside projects.project_json.

SQLite development workflow

A separate database for local experiments

Default: state/atlas-ran-twin.sqlite3. Set ATLAS_DB_PATH for a separate development database.

Read-only inspection

Open SQLite with mode=ro. Inspect schema, revision and project-scoped run counts.

A consistent backup

Use SQLite backup support for a running WAL database. Test the restored copy.

A tested schema change

Update ProjectStore initialization, compatibility logic and storage/API tests together.

Local HTTP contracts

Endpoint group	Payload / result	Development contract
GET /api/workspace	revision + workspace	Hydrate the persisted owner
PUT /api/workspace	revision, workspace, artifacts	200 confirmed, 400 rejected, 409 conflict
GET /api/projects/{id}/...	tasks, artifacts or runs	Project-scoped lists and pagination
GET /api/runs/{runId}	input, result and events	Retained evidence after restart
GET /api/rt/capability	Runtime availability + actions	Detect availability before execution
POST /api/rt/jobs	Captured job + projectId	202 accepted with retained run record
POST /api/rt/jobs/{id}/... cancel or retry	{ projectId }	Retry the retained inputs

The API enforces local-client and Origin checks. Database files stay outside static serving.

Measurement evidence dependencies

ATLASRAN

DIGITAL TWIN STUDIO

RAN Twin - City Pilot

Example City - Central cluster - 1 project

Projects

Workspace preset

All workspaces

Overview

Data and Twin Setup

Study definition

Measurement datasets

RAN topology

Network Design

Simulation and Experiments

Validation and Optimization

Operations and Reports

Platform settings

5G RAN / Data and Twin Setup / Measurement datasets

Planning workspace

Active project RAN Twin - City Pilot

Study definition v1 - ALL

Working network - Dataset v2 - second.csv - 3 GPS samples - unknown - unverified

Comparison reference

Partial source baseline

Study & scenarios

Measurement datasets

Import GPS CSV

Choose File

No file chosen

DATA AND TWIN SETUP / MEASUREMENTS

Measurement datasets

Retain source versions, review measurement quality, then choose evidence for the working network.

Retained versions

2 / 20

Working dataset

second.csv

Maps and analysis use the working selection. Frozen baselines and run inputs retain their original evidence.

Source versions

Review a version before changing working analysis.

v2

second.csv

3 GPS samples - unknown - unverified

Acquired unknown - retained 10/2/2026, 11:00:00 AM

Working

v1

vendor.csv

3 GPS samples - unknown - unverified

Acquired unknown - retained 10/2/2026, 11:00:00 AM

Clear working selection

RETAINED VERSION 1

vendor.csv

unknown - unverified - 3 GPS samples

Use for working analysis

Open measurement analysis

Download original CSV

KPI availability

KPI	Available	Missing
RSRP / SS-RSRP dBm	2 / 3	1
RSRQ / SS-RSRQ dB	0 / 3	3
SINR / SS-SINR dB	0 / 3	3
DL throughput Mbps	2 / 3	1
UL throughput Mbps	0 / 3	3

Import review

Map columns and units, declare origin/date and inspect missing KPIs.

Retained version

Keep original CSV, normalization recipe and source identity.

Working selection

Share one dataset across analysis and map workspaces.

Frozen studies and candidates

ATLASRAN

DIGITAL TWIN STUDIO

Gangnam SKT registered sites · synthetic drive

Seoul · Gangnam · Nonhyeon-dong / Teheran-ro · Gangnam · 1 project

Projects

Workspace preset

All workspaces

Overview

Data and Twin Setup

Network Design

City map

Site & cell planner

Radio planner

Baselines & candidates

Simulation and Experiments

Validation and Optimization

Operations and Reports

Platform settings

5G RAN / Network Design / Baselines & candidates

Planning workspace

Active project

Gangnam SKT registered sites · synthetic drive

Study definition v1 · NR

Working network · Legacy dataset · gangnam-nonhyeon-teheran-synthetic-drive.csv · 676 GPS samples · synthetic · unverified

Comparison reference

Street tilt candidate v2

Study & scenarios

Measurement datasets

ENGINEERING STUDY

Study & scenarios

Review working network

Define the decision, freeze the network inputs, then review candidate RF changes against that baseline.

Definition & evidence

Baselines & candidates

Study reference saved

Capture baseline inputs

Create a candidate scenario

Baseline library

1 snapshots

BASELINE	SCOPE / EVIDENCE	CONTENT IDENTITY	REFERENCE
Gangnam baseline 10/2/2026, 8:55:42 AM	Definition v1 · 9 sites 676 GPS samples · origin unknown	7f8540096691...	Reference Gangnam baseline

Candidate review

Street tilt candidate · v2

Baseline Gangnam baseline · definition v1 · candidate v2 · 1 RF changes

Working network matches the captured inputs.

Issue / hypothesis: Review weak samples near Nonhyeon-dong

Historical revision is read-only. Select the latest revision to continue editing.

Candidate site

RF parameter

Candidate value

Definition history

Record the decision, RAT, carrier and review window.

Baseline and revision

Retain inputs and append changes with content digests.

Execution support

Candidate input edits must fit the current solver’s supported model.

Run lifecycle and recovery

Overview

Data and Twin Setup

Network Design

Simulation and Experiments

Ray tracing lab

Jobs & runs

Virtual UE fleet

Package A/B test

AI-RAN data generation

Validation and Optimization

Operations and Reports

Platform settings

Inspect local path execution, retained input versions and recovery history for RAN Twin · City Pilot.

Recorded local Sionna-RT path jobs use uncalibrated geometry and isotropic antennas. Work-plan dates and tasks do not schedule this solver.

Recorded path runs

Refresh run list

Server-recorded Sionna-RT jobs for RAN Twin · City Pilot. Planned delivery tasks remain unexecuted.

1 run recorded · showing 1

Run 35aedc72

10/2/2026, 8:55:34 AM · sionna-rt

CANCELLED

Path count unavailable

Selected run

CANCELLED

Run 35aedc72-deb3-45a7-9c55-c5016e35ba83

Cancelled by user; worker stopped and output discarded

Working-network capture

Network inputs changed since this run. Retained output still describes its original inputs.

Definition v1 · dataset synthetic · seed 42

Network inputs

Job settings

e7d5456dd07230aa...

2380403e8b0e22d0...

Input identity identifies a saved configuration. Geometry, materials and antenna assumptions remain uncalibrated.

Worker stopped; output discarded. The frozen request remains in run history.

Retry frozen inputs

Reuse the original inputs, receiver, options and seed in a new linked run. Working-network edits are excluded.

Run lifecycle · 4 events

10/2/2026, 8:55:34 AM	queued	Frozen path request queued
10/2/2026, 8:55:34 AM	running	Worker started
10/2/2026, 8:55:34 AM	cancelling	Cancellation requested; awaiting worker exit

Retained lifecycle

SQLite stores the captured request, result and ordered events.

Cancellation

Keep the worker slot until the worker has stopped.

Exact retry

Create a linked run from frozen inputs, excluding working-network edits.

Optional Sionna-RT environment

`npm run rt:setup`
`npm run test:rt`
`npm start`

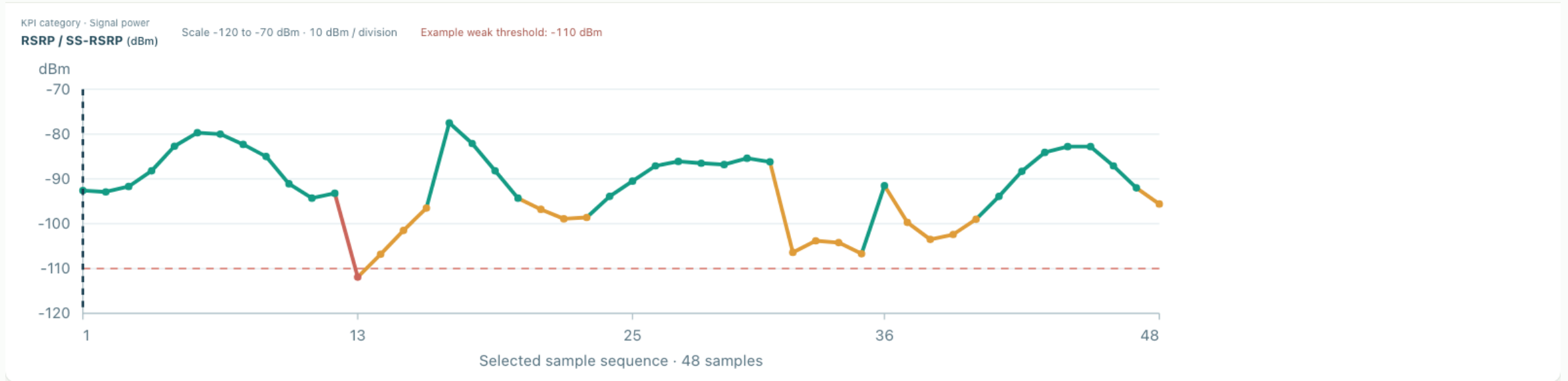
Separate Python 3.13 environment
Sionna-RT 2.1.0
Working native CPU/GPU backend

Bound	Current worker contract
Input geometry	Up to 100 WGS84 building footprints
Solver request	Depth 0–4, 1,000–200,000 samples per source
Execution	One active worker, bounded timeout
Result semantics	Single-link paths with uncalibrated material/antenna assumptions

Extension points and code dependencies

Change area	Primary code	Related validation
KPI chart / analysis	DrivePreviewLeaf.tsx dm.mjs + dm-trend.mjs + dm-ui.mjs	Unit, component and drive browser tests
Site / cell editing	SiteInspector.tsx sitePlannerCommands.ts + radioCommands.ts	Model, command and planner tests
CSV evidence	driveImport.ts + driveImportCommands.ts measurement-library.mjs	Replay, digest and Python validators
Frozen studies	study.mjs + study-run.mjs src/features/study/	JS/Python transition and capture contracts
Local path execution	rt-job.mjs + rt_worker.py serve.py JobManager	Lifecycle/API tests and optional real RT
Reports / artifacts	artifacts.mjs src/features/artifacts/	Generated-content and save parity tests

Worked example: the KPI trend chart



Responsibility	Files to change
Metric metadata and exact axes	dm.mjs + dm-trend.mjs
Both presenters and responsive layout	DrivePreviewLeaf.tsx + dm-ui.mjs + dm.css
Behavior and visual regressions	dm-trend.test.mjs; DrivePreviewLeaf.test.tsx Browser: drive-kpi-trend.spec.ts

Adding a persisted feature

A field inside `project_json` can still require validation and compatibility changes across languages

Change	Implementation responsibility
Domain contract	Add defaults, bounds, upgrade behavior and export semantics
UI and command adapter	Keep draft state local and dispatch a project-scoped change
Transport contract	Parse the envelope and validate domain/evidence content
Python storage validation	Reject invalid or unsupported persisted claims
Historical evidence	Preserve captured studies, measurement records and terminal runs
Tests and graph	Exercise save/reload/conflict paths and refresh Graft

Test layers

Layer	Command	What it establishes
Domain + Python API	<code>npm test</code>	Root <code>.test.mjs</code> plus selected unittest suites
Strict types	<code>npm run typecheck</code>	TypeScript contracts and component call signatures
Component / API contracts	<code>npm run test:ui</code>	Vitest under jsdom
Production / Vite integration	<code>npm run test:integration</code>	Build, serving, save and conflicts
Real browser workflows	<code>npm run test:e2e</code>	Chromium navigation, input and visual behavior
Optional real solver	<code>npm run test:rt</code>	Installed Sionna-RT backend execution
Combined local gate	<code>npm run check</code>	Tests, types, components, build and syntax

Focused validation commands

```
node --test dm-trend.test.mjs
```

```
npm run test:ui -- src/features/drive/DrivePreviewLeaf.test.tsx
```

```
npm run test:e2e -- tests/e2e/drive-kpi-trend.spec.ts
```

Full local gate

```
npm run check
```

Browser installation

```
npm exec --no -- playwright install  
chromium
```

Validation snapshot: 2 October 2026

Stage	Observed result	Status
JavaScript domain tests	124 passed	Passed
Python tests	38 passed	Passed
Strict TypeScript	SiteInspector / RadioField signature mismatches	Failed
Component tests	Sequential gate stopped before this stage	Not run
Production integration / build	Sequential gate stopped before this stage	Not run
Browser suite / real RT	Separate commands, outside this validation run	Not run

Resolve the reported call/type mismatch and repeat the gate before treating this snapshot as ready.

Cross-platform CI configuration

Configuration	Current workflow
Operating systems	windows-latest and ubuntu-latest
Toolchain	Node 24 and Python 3.13
Path compatibility	Checkout directory: atlas workspace
Install	npm run setup
Main gate	npm run check
Browser gate	Install Chromium with OS dependencies, then test:e2e

These are workflow settings. Confirm the actual run result in CI for each change.

Troubleshooting

Symptom	Check	Repair direction
Connection refused	Printed URL and server process	Start the correct server or use its selected port
Vite API request fails	ATLAS_API_ORIGIN matches Python port	Restart Vite with the correct local target
SQLite cannot open	ATLAS_DB_PATH and parent directory access	Use an accessible intended database location
Save conflict / dirty draft	Server revision and comparison view	Reconcile the draft through the controller
Missing native binding	Node architecture and lockfile binding	Install platform-specific optional dependencies
UI change absent	Source checkout, dist build and browser cache	Rebuild the production bundle and reload

Additional development guide

Next development area	Required code / evidence work
Stabilize the current branch	Resolve type errors, complete local and browser gates
Broader radio-model support	Represent antenna/cell settings in the solver and test captures
Calibration and comparison	Align nonsynthetic evidence and retain comparable completed runs
Connected observation	Add authenticated adapters with timestamped source status
Controlled changes	Implement a reviewed change adapter and rollback contract

For each feature: identify its state owner, trace dependencies, preserve evidence, validate the result and refresh Graft.